

420-126-LI

POO 1

Variables et méthodes simples

Plan

- Variables (principe de base)
- Nommage des variables (règles et conventions)
- Déclaration, affectation (ou modification d'une variable)
- Traces d'exécution
- Introduction au type «String»
- Affichage d'une variable à la console
- Structuration de programme avec des méthodes simples
- Méthodes simples (sans valeur de retour et sans paramètres)

Variables (principe de base)

- Les variables sont des symboles qui associent un nom (un identifiant) à une valeur. La valeur peut être d'un certain type de donnée qui doit être choisi une fois pour toutes.
- On peut voir une variable comme **une boîte** dans laquelle on peut placer **une donnée d'un certain type**.

Exemple

Type (ici un entier)

Nom



int

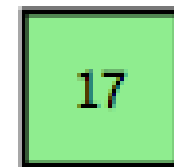


variable;

variable = 17;

← Valeur ici 17

int



variable

- On verra qu'il existe bien d'autres types que `int` (`double`, `string`, `bool` ...)

Déclaration, affectation (ou modification d'une variable)

```
// Déclaration des variables
int premierNombre;
int secondNombre;
int resultat;

// Affectation des variables
premierNombre = 5;
secondNombre = 8;

// On écrase la valeur 5 par 12 dans premier nombre
premierNombre = 12;

// Opération sur des variables
resultat = premierNombre + secondNombre;
```

- Les **déclarations** de variable se font au début du bloc de code par convention.
- Les **affectations** se font lorsqu'on en a besoin. Une nouvelle affectation remplace la valeur précédente... Cette dernière n'est plus accessible!
- **Très bonne pratique** : On peut combiner déclaration et affectation afin de déclarer et initialiser une variable. (L'initialisation est la première affectation)

```
int autreNombre = 6;
```

Nommage des variables (règles et conventions)

- Un nom de variable ne peut comporter que des lettres et des chiffres
- Les noms de variables ne peuvent pas être les **noms réservés du langage**
- Un nom de variable ne peut pas commencer par un chiffre et ne doit pas comporter d'espace
- Le langage Java est sensible à la casse, ce qui signifie que des lettres majuscules et minuscules ne constituent pas la même variable.
- Le **nom d'une variable** est un **identificateur unique** pour une portion de code. Il faudra choisir un nom qui décrit bien son contenu
- les noms de variables commencent par une minuscule et ne comportent ni « _ », ni « \$ »
- contiennent des majuscules uniquement à chaque changement de mot (on appelle cela le « camelCase »)
- on n'utilise **pas d'accent**. On préfère **eleve** (« élève » fonctionne, mais **ne doit pas** être utilisé)
- Dans la mesure du possible on évite les abréviations qui peuvent introduire un doute

Nommage des variables (exemples)

Explication du contenu de la variable	Bon nom de variable
Vitesse d'une voiture	
Nombre de réussites	
Somme des factures	
Nom de l'employé	
Vrai si c'est un succès	
Vrai si c'est réussi	

Nommage des variables (exemples)

Explication du contenu de la variable	Bon nom de variable
Vitesse d'une voiture	vitesseVoiture
Nombre de réussites	nbrReussite ou nombreReussite
Somme des factures	sommeFacture
Nom de l'employé	nomEmploye
Vrai si c'est un succès	succes
Vrai si c'est réussi	reussi

Traces d'exécution

- On appelle une trace d'exécution, un tableau qui donne l'historique des valeurs des différentes variables durant l'exécution d'un programme. Cet outil est très intéressant pour vérifier la validité d'un algorithme. (*Algorithme : suite finie et non ambiguë d'opérations ou d'instructions permettant de résoudre un problème*)

```
public class Application{  
    public static void main(String[] args){  
  
        int a;  
        int b;  
        int c;  
        a = 8; // ligne 1  
        b = a - 3; // ligne 2  
        c = a + b * 12; // ligne 3  
        b = a + b + c; // ligne 4  
        a = a / 3; // ligne 5  
    }  
}
```

étape	a (int)	b (int)	c (int)
Ligne 1	8	-	-
Ligne 2	8	5	-
Ligne 3	8	5	68
Ligne 4	8	81	68
Ligne 5	2	81	68

Introduction au type string

- Nous avons vu que pour utiliser des chaînes de caractères en Java, il suffit d'écrire la séquence de caractères entre guillemets doubles. Par exemple, pour afficher le texte **Hello World !** à l'écran, on doit écrire l'instruction suivante :

```
System.out.println("Hello World !")
```

- Pour créer une nouvelle variable chaîne de caractères, on fera comme tous les types de base en Java :

```
String maChaine = " Hello World !";
```

Introduction au type string

On peut concaténer des chaînes de caractères à l'aide de l'opérateur de concaténation « + », comme ici:

```
String prenom = "Julien";  
String nom = "Brunet";  
String prenomEtNom = prenom + " " + nom;  
System.out.println (prenomEtNom);  
System.out.println ("J'ai " + 49 + " ans.");
```

- À l'exécution de cette portion de code, voici ce qui sera affiché :

```
Julien Brunet  
J'ai 49 ans.
```

Affichage d'une variable à la console

```
int maintenant = 2023;  
int anneeTitanic = maintenant - 1912;  
System.out.println ("Le Titanic a coulé en 1912.");  
System.out.println ("Si nous sommes en " + maintenant + ".");  
System.out.println ("Ça fait " + anneeTitanic + " ans qu'il a coulé.");
```

On remarque que L'opérateur de concaténation « + » permet aussi de **convertir** des nombres en chaines de caractères sans effort ou instruction particulière.

Le résultat est

```
Le Titanic a coulé en 1912.  
Si nous sommes en 2023.  
Ça fait 111 ans qu'il a coulé.
```

Méthodes simples (sans valeur de retour et sans paramètres)

- Une déclaration de méthode doit respecter la syntaxe:

```
public void nomDeLaMethode() {  
    //Mettre ici le code de la méthode  
}
```

Exemple:

```
public void bonjourLeMonde() {  
    System.out.println("Bonjour tout le monde!");  
}
```

- Si on ne l'appelle pas explicitement ce code ne sera **jamais exécuté**
- Pour pouvoir l'exécuter il faut se fabriquer un objet de la classe dans laquelle la méthode est définie:

Exemple5 **monProgramme** = **new** Exemple5();

- Pour l'exécuter on utilise l'objet créé pour faire un appel de méthode par exemple:

monProgramme.bonjourLeMonde();

- Une méthode peut être appelée autant de fois que souhaité à plusieurs endroits différents!
- Cela permet de **factoriser (diviser), structurer et de réutiliser!**

Structuration des méthodes sur des fichiers avec des

- Ouvrez le fichier **Exemple5.java** et observons:

```
package semaine1;

public class Exemple5 {
    //déclaration d'une méthode appelée afficherBonjourLeMonde
    public void afficherBonjourLeMonde() {
        System.out.println("Bonjour le monde!");
    }

    //déclaration d'une méthode appelée afficher1Plus1
    public void afficher1Plus1() {
        System.out.println("");
        System.out.print("1 + 1 = ");
        System.out.println(1 + 1);
    }

    //déclaration de la méthode main, c'est le point d'entrée du programme
    public static void main(String[] args) {
        System.out.println("Exécution du main de la classe Exemple5");
        //instanciation de la classe Exemple5, encore un peu
        Exemple5 programme = new Exemple5();
        //Appel de la méthode afficherBonjourLeMonde sur l'objet appelé programme
        //c'est ce qui va provoquer l'exécution du code de la méthode
        programme.afficherBonjourLeMonde();
        //autre appel de méthode
        programme.afficher1Plus1();
        //Une méthode peut être appelée plusieurs fois!!
        programme.afficherBonjourLeMonde();
    }
}
```

Force le fichier à se trouver dans un répertoire semaine1 dans « src »

2 déclarations de méthode qui contiennent du code « prêt être exécuté ».

Le point d'entrée du programme appelé le **main**. C'est par ici que le programme va commencer son exécution et donc que vous devez en commencer la lecture!!

Fabrique un objet (on dit « instancier un objet de la classe Exemple5 »), ce qui est nécessaire pour accéder au code des méthodes de cette classe.

Les appels de méthode sur l'objet nommé « programme » vont générer l'exécution du code qu'elles contiennent.